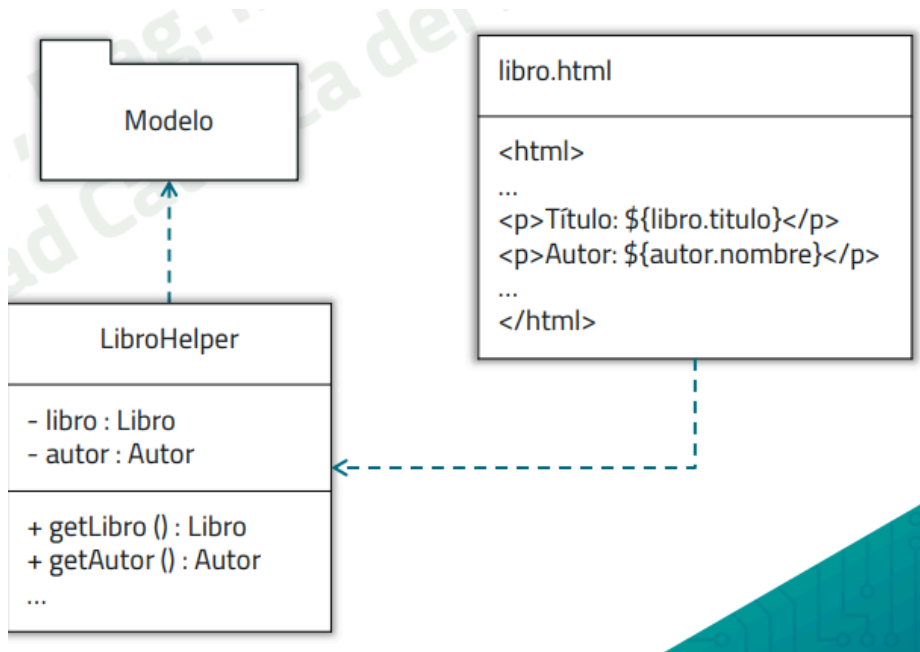


1. Patrones de Presentación

Template View



Definición: Embebe marcadores en páginas HTML estáticas que se reemplazan por datos dinámicos al cargar.

Características:

- Marcadores como `${libro.titulo}` en HTML
- Separación entre diseñador gráfico y programador
- Ejemplos: JSP, Thymeleaf, Handlebars

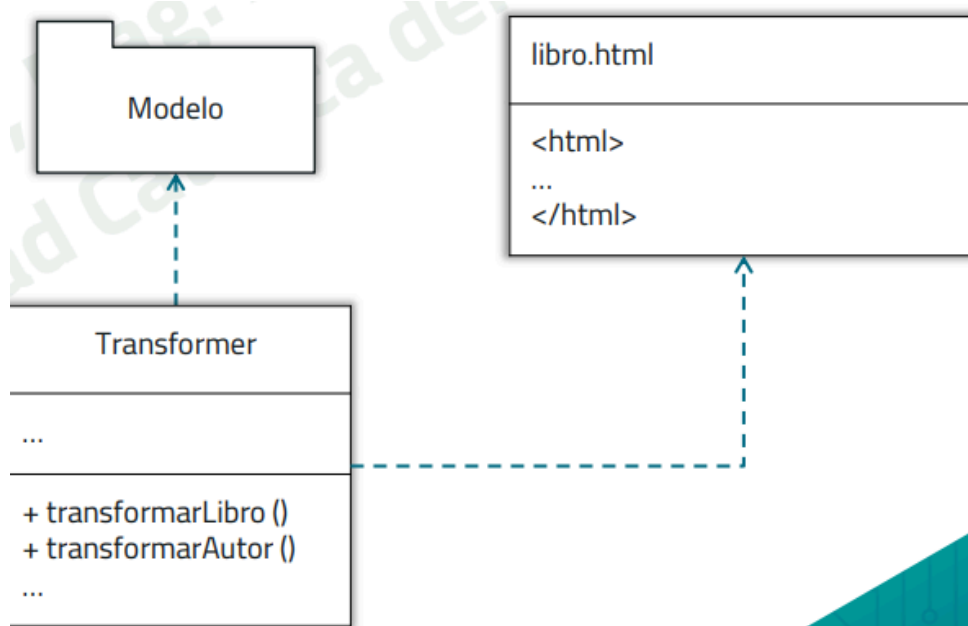
Ventajas:

- ☒ Fácil entendimiento del código
- ☒ Separación clara de responsabilidades

Desventajas:

- **✗** Riesgo de incluir lógica compleja en plantillas
 - **✗** Requiere servidor web para pruebas
-

Transform View



Definición: Clase transformadora que convierte datos de dominio a HTML.

Características:

- Clase especializada en transformar datos a HTML
- Lógica de renderización separada
- Ejemplo: Transformer que genera HTML desde objetos

Ventajas:

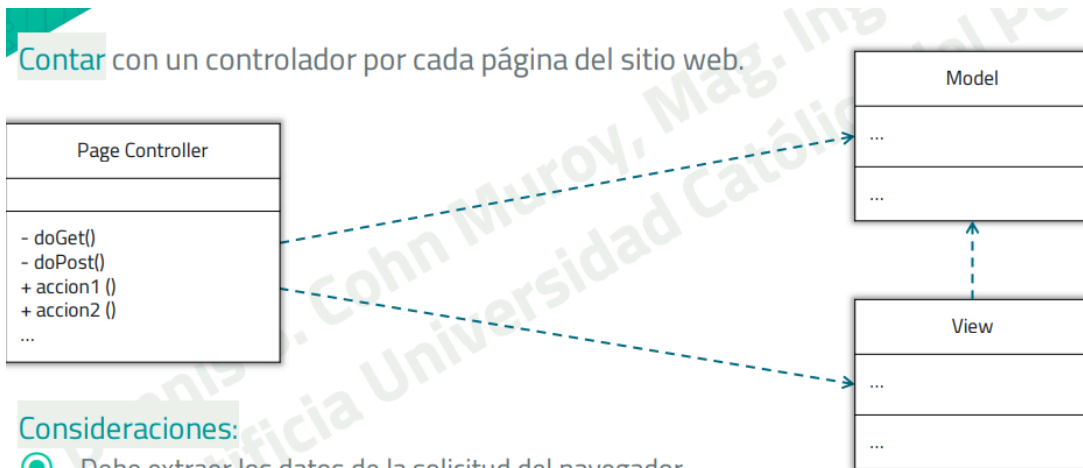
- ☒ No requiere servidor web para pruebas
- ☒ Enfoque único en renderización

Desventajas:

- ☒ Modificar apariencia requiere cambiar múltiples clases
- ☒ Más complejo de mantener

2. Patrones de Controlador

Page Controller



Definición: Un controlador específico para cada página del sitio web.

Características:

- Un controlador por página/vista
- Maneja solicitudes GET/POST específicas
- Responsabilidades:
 - Extraer datos de solicitud

- Invocar objetos del modelo
- Determinar vista a mostrar

Ventajas:

- ☒ Acciones particulares atendidas por controladores específicos
- ☒ Eficaz para lógica simple

Desventajas:

- ☒ Fuerte acoplamiento controlador-vista
- ☒ Muchos controladores en proyectos grandes
- ☒ Complejidad de mantenimiento

¿CUÁNDO USAR PAGE CONTROLLER?

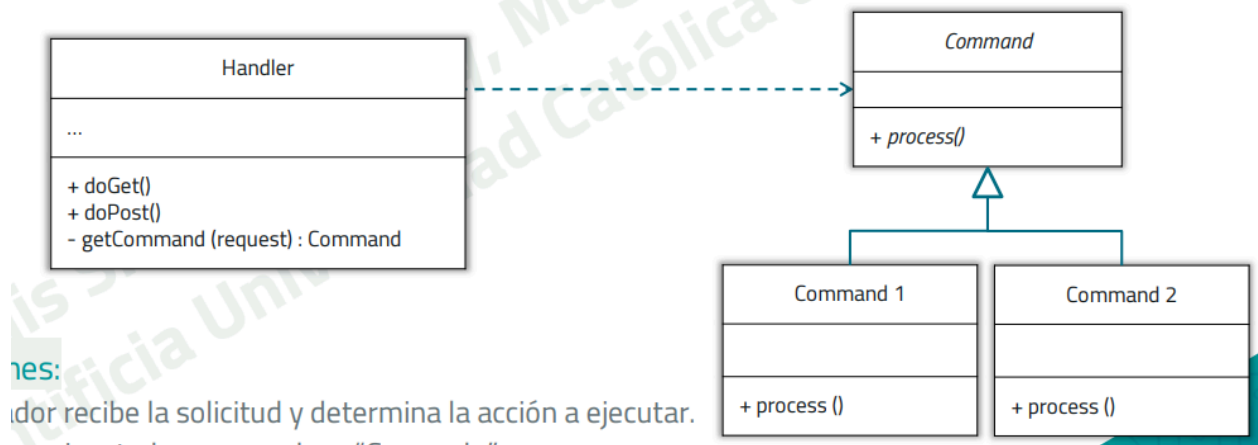
Ventajas:

- Acciones particulares son atendidas por determinadas vistas y controladores.
- Eficaz cuando la lógica del controlador es simple.

Desventajas:

- Fuerte acoplamiento entre el controlador y su vista.
- Mayor complejidad de mantenimiento en proyectos grandes dado el alto número de controladores.

Front Controller ★



nes:
dor recibe la solicitud y determina la acción a ejecutar.

Definición: Un único controlador que gestiona todas las llamadas al sitio web.

Características:

- Patrón MVC tradicional
- Componentes:
 - Front Controller: Recibe todas las solicitudes
 - Handler: Determina acción a ejecutar
 - Command: Ejecuta la acción específica
 - View: Determina qué vista mostrar

Flujo:

text

Solicitud → Front Controller → Handler → Command → View

Ventajas:

- ☒ Funcionalidad extensible mediante nuevos Commands
- ☒ Código común consolidado en clase padre
- ☒ Permite decoradores: autenticación, i18n, encoding
- ☒ Arquitectura más mantenible

Desventajas:

- ☒ Posible cuello de botella
- ☒ Mayor complejidad en controlador centralizado

Ventajas:

- ☒ Funcionalidad extensible a nivel de nuevos "Comandos".
- ☒ Permite consolidar código común en la clase padre "Comando".
- ☒ Permite adecuar su comportamiento a través de decoradores: autenticación, internacionalización, codificación, etc.

Desventajas:

- ☒ Posible cuello de botella.
- ☒ Mayor complejidad al mantener un controlador centralizado.

3. Guía de Selección

¿Cuándo usar cada patrón?

Template View vs Transform View:

- Template View: Cuando hay diseñadores gráficos en el equipo
- Transform View: Cuando el equipo es principalmente de desarrollo

Page Controller vs Front Controller:

- Page Controller: Aplicaciones pequeñas con lógica simple

- Front Controller: Aplicaciones empresariales complejas

4. Recomendaciones Arquitectónicas

Mejores Prácticas:

1. Nunca incluir lógica de negocio en las vistas
2. Nunca hacer llamadas a BD desde vistas o controladores
3. Separar claramente: Modelo ↔ Controlador ↔ Vista
4. Usar Front Controller para aplicaciones que escalarán

Arquitectura Recomendada para Empresas:

text

Front Controller → Command Pattern → Business Layer → Data Layer

↓

Template View/Transform View

Patrón	Tipo	Complejidad	Escalabilidad	Caso Ideal
Template View	Presentación	Baja	Media	Equipos con diseñadores
Transform View	Presentación	Media	Alta	Equipos de desarrollo
Page Controller	Controlador	Baja	Baja	Apps pequeñas
Front Controller	Controlador	Alta	Alta	Apps empresariales

Para sistemas web empresariales: La combinación Front Controller + Template View ofrece el mejor balance entre mantenibilidad y separación de responsabilidades.